

Industrial Oriented Mini Project (CM653PC)

On

“HELIOS -AN AUTONOMOUS DESKTOP AGENT”

Submitted

In the partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

In

Computer Science Engineering

(Artificial Intelligence & Machine Learning)

by

Mohammed Abdul Razzak (23265A6606)

and

Bharath M (24265A6604)

Under the guidance of

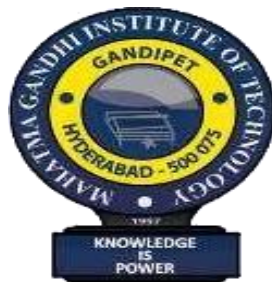
Ms. Karuna Verma

(Assistant Professor)

and

Mr. Sunil Kumar

(Assistant Professor)



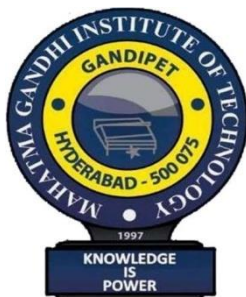
Department of Emerging Technologies
MAHATMA GANDHI INSTITUTE OF TECHNOLOGY(Autonomous)
(Affiliated to Jawaharlal Nehru Institute of Technological University Hyderabad)
Kokapet(V), Gandipet(M), Hyderabad.
Telangana -- 500 075
2025-2026

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY(Autonomous)

(Affiliated to Jawaharlal Nehru Technological University Hyderabad)

Gandipet, Hyderabad- 500 075, Telangana (India)

CERTIFICATE



This is to certify that the Industrial Oriented Mini Project (CM653PC) entitled "**HELIOS -AN AUTONOMOUS DESKTOP AGENT**", is being submitted by **Mr. Mohammed Abdul Razzak** bearing Roll No: **23265A6606** and **Mr. Bharath M** bearing Roll No: **24265A6604** in partial fulfillment for the award of degree of **Bachelor of Technology in Computer Science and Engineering - Artificial Intelligence And Machine Learning** to **Mahatma Gandhi Institute of Technology** is a record of bona-fide work carried out by them under our guidance and supervision.

The results embodied in this project have not been submitted to any other University or Institute for the award of any degree or diploma.

PROJECT GUIDE

HEAD OF THE DEPARTMENT

Ms. Karuna Verma

Assistant Professor, Dept of ET

Prof. M. Rama Bai

Professor, Dept of ET

Mr. K. Sunil Kumar

Assistant Professor, Dept of ET

External Examiner

DECLARATION

This is to certify that the work reported in this project titled "**HELIOS –AN AUTONOMOUS DESKTOP AGENT**" is a record of work done by us in the Department of Computer Science and Engineering (Artificial intelligence and Machine Learning), Mahatma Gandhi Institute of Technology, Hyderabad.

No part of the work is copied from books/journals/internet and wherever the portion is taken, the same has been duly referred in the text. The report is based on the work done entirely by us and not copied from any other source.

MOHAMMED ABDUL RAZZAK
23265A6606

BHARATH M
24265A6604

ACKNOWLEDGEMENTS

We would like to express my sincere thanks to **Dr. G. ChandraMohan Reddy, Principal MGIT**, for providing the working facilities in college.

We wish to express my sincere thanks and gratitude to **Dr. M. Rama Bai, Professor and HoD, Department of ET, MGIT**, for all the timely support and valuable suggestions during the period of project.

We are extremely thankful to our Project Coordinators **Ms. J. Sreedevi, Assistant Professor** Department of ET, MGIT, industrial oriented mini project coordinators for their encouragement and support throughout the project.

We are extremely thankful and indebted to my internal guide **Ms. Karuna Verma, Assistant Professor, Mr. M. Sunil Kumar, Assistant Professor**, Department of ET, MGIT for their constant guidance, encouragement and moral support throughout the project.

Finally, we would also like to thank all the faculty and staff of ET Department who helped us directly or indirectly, for completing this project.

MOHAMMED ABDUL RAZZK
23265A6606

BHARATH M
24265A6604

ABSTRACT

In the era of artificial intelligence, the human computer interaction paradigm is shifting from rigid, click-based operations to natural, conversational control. Despite the rapid progress of cloud-based assistants such as Cortana, Siri and Google Assistant, a genuinely autonomous, privacy-preserving desktop agent that can operate primarily offline and control a full Windows environment through natural language is still missing. This project presents HELIOS (Hybrid Execution Layer for Intelligent OS-level Supervision), an offline-first autonomous desktop AI agent that understands free-form natural-language commands and executes them on a Windows PC opening applications, managing files, composing emails, scheduling reminders, toggling system settings and performing autonomous web tasks without requiring continuous cloud connectivity.

The proposed methodology combines a locally hosted Large Language Model (Gemma 3 or Mistral-7B served via Ollama) with an optional Gemini 2.0 Flash / GPT-4o-mini cloud fallback through a hybrid routing layer. A Natural-Language Router parses any phrasing of a user command into a structured JSON intent, which the orchestrator (agent.py) dispatches to purpose-built Python modules for desktop automation (PyAutoGUI), system controls (PowerShell, psutil), file management, Gmail composition, task scheduling (APScheduler), note-taking, web search (DuckDuckGo) and chat-history persistence. A floating Tkinter popup provides an always-on-top chat interface with typing animations, session history and theme switching.

The results demonstrate that HELIOS successfully recognises more than thirty distinct intent types across fifty-plus OS-level commands, achieves a response time of 2–8 seconds on a 16 GB RAM machine with the local Gemma3 model, and operates with 95 % of its features fully offline. HELIOS therefore proves that a private, offline-first, autonomous desktop assistant is achievable on commodity consumer hardware.

TABLE OF CONTENTS

TOPIC	PAGE NO.
Certificate	i
Declaration	ii
Acknowledgement	iii
Abstract	iv
List of Figures	vi
List of Tables	vii
1. Introduction	1
1.1 Motivation	2
1.2 Problem Definition	2
1.3 Existing System	2
1.4 Proposed System	2
1.5 Requirements Specification	3
2. Literature Survey	4
3. Design and Methodology	7
3.1 System Architecture	7
3.2 Hybrid LLM Routing	8
3.3 Natural Language Router	9
3.4 Action Modules	10
3.5 Tools and Technologies	11
4. Testing and Results	12
4.1 Testing	12
4.2 Results	13
5. Conclusion and Future Scope	17
BIBLIOGRAPHY	18
APPENDIX	19

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO
Figure 3.1	HELIOS – High-Level System Architecture	7
Figure 3.2	Hybrid LLM Mode Decision Flow	8
Figure 3.3	Natural-Language Router to Module Dispatch	9
Figure 4.1	HELIOS Floating Chat Interface & History Panel	13
Figure 4.2	HELIOS Running in VS Code Sample Offline Session	14
Figure 4.3	Autonomous Swiggy Web Task Pizza Ordering Flow	15

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO
Table 2.1	Literature Survey on HELIOS (Related Works & Repos)	5
Table 3.1	HELIOS Action Modules and Their Responsibilities	10
Table 4.1	HELIOS Performance Metrics Summary	16

INTRODUCTION

PROBLEM STATEMENT:

Modern operating systems expose an enormous surface of features, yet users still interact with them through a rigid sequence of clicks, keyboard shortcuts, and memorized commands. With the rise of Large Language Models (LLMs), it is now possible to build an autonomous software layer that sits between the user and the operating system, understands free-form natural language, and performs complex multi-step tasks on the user's behalf (Brown et al., 2020, Wei et al., 2022).

HELIOS is an offline-first autonomous desktop AI agent designed for Windows. It runs a locally hosted LLM such as Gemma 3 or Mistral-7B via Ollama as its default reasoning engine and falls back to Gemini 2.0 Flash or GPT-4o-mini only when a task explicitly requires internet-scale knowledge (e.g., live news, weather, or stock prices) (Google, 2024 OpenAI, 2024).

The agent exposes a single natural-language chat interface and can open applications, search and move files, compose Gmail drafts, toggle system settings such as Wi-Fi and Bluetooth, schedule reminders, perform web searches using DuckDuckGo, and execute autonomous workflows such as online food ordering. This aligns with emerging agent-based paradigms that combine reasoning and action in language models (Yao et al., 2022).

1.1 Motivation

Three primary factors motivate the development of HELIOS:

- **Productivity** Users lose significant time switching between applications, remembering shortcuts, and searching for files across large storage systems. Intelligent agents can reduce this overhead through task automation.
- **Accessibility** Novice users, elderly individuals, and visually impaired users benefit significantly from conversational interfaces layered on top of traditional operating systems (Hoy, 2018).
- **Privacy** Most commercial assistants such as Siri, Cortana, and Google Assistant rely heavily on cloud processing. HELIOS is designed to be private by default, with approximately 95% of its features operating entirely on-device (Luger & Sellen, 2016).

1.2 Problem Definition

Current desktop environments function primarily as passive tools. Existing voice assistants are either limited in the range of commands they can interpret or are entirely cloud-dependent. Traditional automation approaches such as PowerShell, AutoHotkey, and batch scripting require rigid syntax and break easily when user interfaces or phrasing change.

There is therefore a need for an intelligent system that:

- Understands arbitrary natural-language variations of the same intent
- Plans and executes multi-step tasks autonomously
- Operates primarily offline across the full functionality of the operating system

1.3 Existing System

- **Manual Shortcuts and Scripts** Tools such as PowerShell, AutoHotkey, and batch files are brittle and require exact syntax, limiting usability.
- **Commercial Voice Assistants** Systems like Siri, Cortana, and Google Assistant provide limited OS-level control and depend heavily on cloud infrastructure.
- **Cloud-based LLM Chatbots** Platforms like ChatGPT and Gemini offer powerful reasoning but lack direct control over local systems and raise privacy concerns since all interactions are processed remotely.
- **Local LLM Tools** Frameworks such as Ollama and GPT4All provide offline inference but lack integration with system-level actions, scheduling, and automation.

1.4 Proposed System

HELIOS is a hybrid, modular desktop agent that integrates the strengths of existing systems while addressing their limitations. Its key objectives include:

- **Natural-language control** Converting arbitrary user input into structured JSON intents
- **Offline-first architecture** Default reasoning using local LLMs with minimal cloud reliance
- **Full OS interaction** Managing system settings, files, applications, and scheduling tasks
- **Autonomous web execution** Performing guided workflows such as online ordering
- **Model-agnostic design** Seamless switching between models like Gemma, Mistral, Gemini, and GPT via configuration The system follows modern agent design principles combining reasoning and action, similar to frameworks proposed in recent research (Yao et al., 2022, Xie et al., 2024).

1.5 Requirements Specification

Software Requirements:

- Programming Language: Python 3.10+
- Operating System: Windows 10 / 11 (64-bit)
- IDE: Visual Studio Code
- Local LLM Runtime: Ollama
- Core Libraries:
- PyAutoGUI
- psutil
- APScheduler
- requests, python-dotenv, Pillow, Tkinter
- Cloud APIs (Optional): Gemini 2.0 Flash, GPT-4o-mini

Hardware Requirements

- RAM: 8 GB minimum, 16 GB recommended
- Storage: 20 GB free space (for model weights)
- Processor: Intel Core i5 (8th Gen) / AMD Ryzen 5 or higher
- GPU (Optional): NVIDIA GPU with ≥ 6 GB VRAM for faster inference
- Display: Any standard monitor (adaptive Tkinter interface)

2. LITERATURE SURVEY

Research in autonomous software agents has advanced significantly with the emergence of large language models (LLMs), moving from traditional automation systems toward intelligent agents capable of reasoning, planning, and interacting with operating systems. The development of open-weight foundation models such as Llama 3 and Gemma has played a major role in this transition by enabling high-quality local inference and instruction-following capabilities on consumer hardware [2, 3]. These models have demonstrated that sophisticated reasoning can be achieved without relying entirely on cloud-based services, thereby promoting privacy-preserving and offline-first AI systems. Frameworks such as Ollama further accelerated this trend by providing lightweight infrastructure for deploying and serving quantized language models locally, making practical desktop AI assistants increasingly feasible [5].

A major direction in recent research focuses on autonomous agents that combine reasoning with action execution. AutoGPT demonstrated the potential of recursive planning and tool-based task execution, popularizing the concept of self-directed AI agents capable of performing multi-step objectives [6]. Similarly, the ReAct framework introduced an effective methodology for interleaving reasoning and actions, enabling language models to make decisions while interacting with external tools and environments [8]. These approaches significantly improved agent autonomy and task completion capabilities. However, many early agent systems relied heavily on cloud-hosted models, incurred high operational costs, and often suffered from looping behaviors and limited control mechanisms. Consequently, researchers have increasingly explored more efficient and controlled architectures that balance autonomy with reliability.

Operating system-level automation has emerged as another important research area. Systems such as UFO² Desktop AgentOS extend agent capabilities beyond text-based environments by enabling direct interaction with desktop applications through multimodal understanding and multi-agent collaboration [1]. While such systems demonstrate impressive automation capabilities, they often depend on cloud-based vision models and computationally expensive infrastructures. Benchmarking initiatives such as OSWorld further highlight the growing need for intelligent agents capable of handling diverse real-world computer tasks across applications and operating systems

[4]. However, these benchmarks primarily focus on evaluation rather than providing lightweight deployable solutions for end users.

Several supporting technologies have contributed to the practical implementation of autonomous desktop agents. GUI automation frameworks such as PyAutoGUI provide low-level interaction primitives including mouse control, keyboard automation, screenshot capture, and screen element detection, forming the foundation for operating system interaction [9]. Scheduling frameworks such as APScheduler enable autonomous execution of recurring and time-based tasks, extending agent functionality beyond immediate user requests [10]. In addition, privacy-oriented web search solutions such as duckduckgo_search offer keyless information retrieval while minimizing user tracking and data collection [7]. Although these tools are powerful, they are not intelligent by themselves and require integration with a reasoning layer to perform meaningful autonomous actions.

Despite substantial progress in the field, several challenges remain unresolved. Many existing autonomous agents depend on cloud services, require expensive API access, consume significant computational resources, and provide limited support for privacy-preserving offline operation. Furthermore, operating system automation frameworks often lack lightweight deployment mechanisms suitable for everyday users. HELIOS addresses these research gaps by integrating local LLM inference, reasoning-action workflows inspired by ReAct, operating system automation through PyAutoGUI, privacy-focused web search, and autonomous task scheduling within a unified desktop assistant architecture. By combining these technologies into a modular offline-first framework, HELIOS seeks to provide practical, secure, and efficient autonomous assistance while reducing dependence on external cloud infrastructure.

Table 2.1 Literature Survey on HELIOS (Related Works & Repositories)

S.NO	REF	AUTHOR / YEAR	TITLE	FOCUS	LIMITATIONS
1	[1]	Microsoft Research, 2025	UFO ² :The Desktop AgentOS	Proposes a multi-agent Windows desktop OS layer that uses GPT-4V for screen understanding.	Cloud-dependent; requires a vision-capable paid API; no offline fallback.
2	[2]	Meta AI, 2024	Llama3:Technical Report	Open-weight LLM family with strong instruction following and tool-use.	Large model sizes; requires local quantisation for consumer.

3	[3]	Google DeepMind, 2024	Gemma: Technical Report	Lightweight open models (2B / 7B / 27B) with strong reasoning for on-device use.	Still compute-intensive; lacks native OS-automation tooling out of the box.
4	[4]	Xie et al. / OSWorld Team, 2024	OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks	Benchmark of 369 real computer tasks across OS, browsers and applications.	Focused on evaluation, not a deployable offline user agent.
5	[5]	Ollama Project, 2023–present	Ollama: Local LLM Serving Framework	Simple one-command serving of quantized open-weight LLMs (Gemma, Mistral, Llama 3) on consumer PCs.	No OS-level tooling out of the box; pure inference server HELIOS wraps it with an NL router and modules.
6	[6]	Significant Gravitas, 2023	AutoGPT: An Autonomous GPT-4 Experiment	Demonstrated recursive self-prompting for multi-step task execution; popularised the idea of tool-using agents.	Cloud-only (GPT-4), expensive, no desktop control, prone to looping.
7	[7]	DuckDuckGo Search Maintainers, 2022–present	duckduckgo_search (Python API)	Keyless, privacy-respecting web search; returns titles, URLs and snippets.	Rate-limited on heavy use; no built-in summarisation HELIOS summarises results via the local LLM.
8	[8]	Yao et al., 2022	ReAct: Synergizing Reasoning and Acting in Language Models	Interleaves reasoning traces with tool-use actions to solve interactive tasks.	Benchmarked on text environments only; no Windows / OS integration.
9	[9]	Al Sweigart, 2019	PyAutoGUI Automate the Boring Stuff with Python	Cross-platform GUI automation primitives clicks, key presses, screenshots, image location on screen.	Not intelligent on its own; requires a reasoning layer (HELIOS LLM engine) to be useful for arbitrary tasks.
10	[10]	Alex Grönholm, 2014–present	APScheduler (Advanced Python Scheduler)	In-process job scheduler supporting date, interval and cron triggers; used by HELIOS for reminders and timed tasks.	No native Windows toast support HELIOS wraps the callback with win10toast.

3. DESIGN AND METHODOLOGY

3.1 System Architecture

HELIOS is organised as a four-layer pipeline: (1) a Tkinter interface layer that exposes a floating chat popup and a session-history side panel; (2) an orchestrator (agent.py) that dispatches parsed intents (3) a set of action modules desktop, system, notes, scheduler, web / Gmail each encapsulating one OS-surface; and (4) a core containing the NL router and the hybrid LLM engine.

Figure 3.1 shows the full architecture.

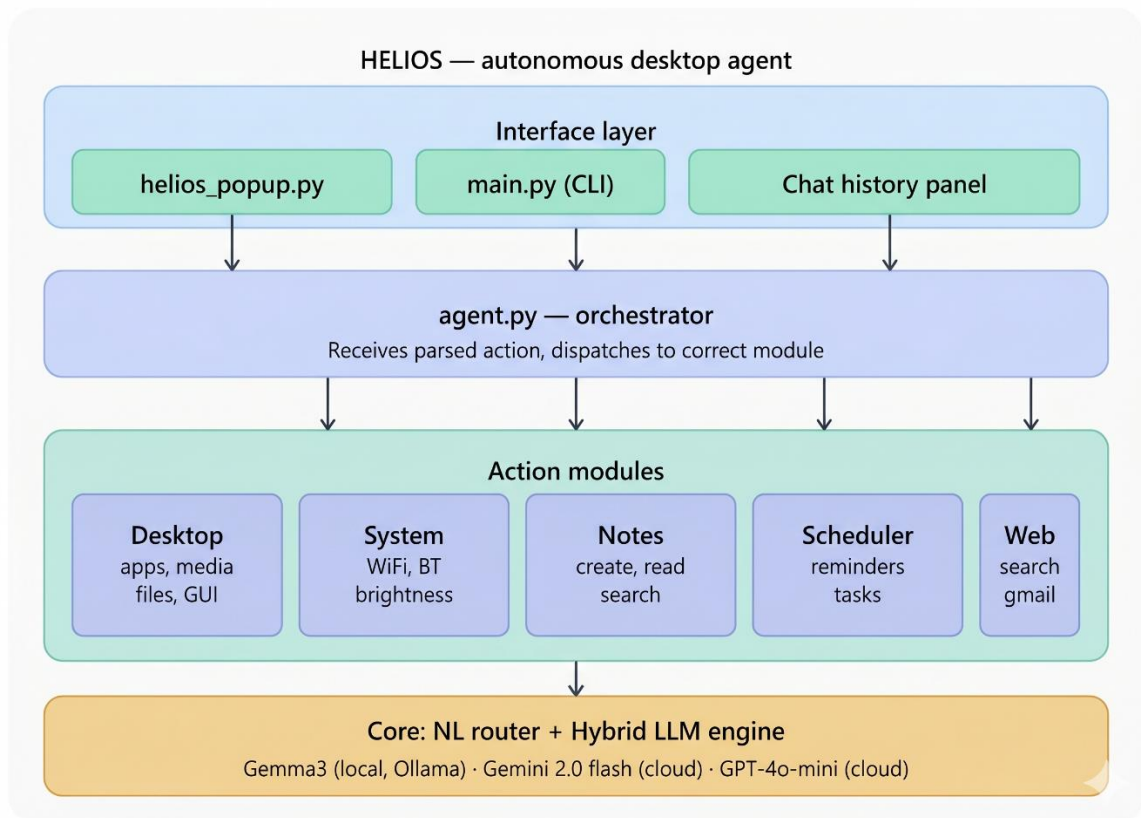


Figure 3.1 HELIOS – High-Level System Architecture

3.2 Hybrid LLM Routing Engine

Every user command enters the LLM router first. The router inspects the LLM_MODE flag in the .env file (offline / auto / online) together with keywords in the command to decide whether to answer locally with Gemma 3 or route to the cloud (Gemini 2.0 Flash by default, GPT-4o-mini as second fallback). Figure 3.2 captures the decision flow note that Gemma 3 is always the final safety net, so HELIOS never completely fails offline.

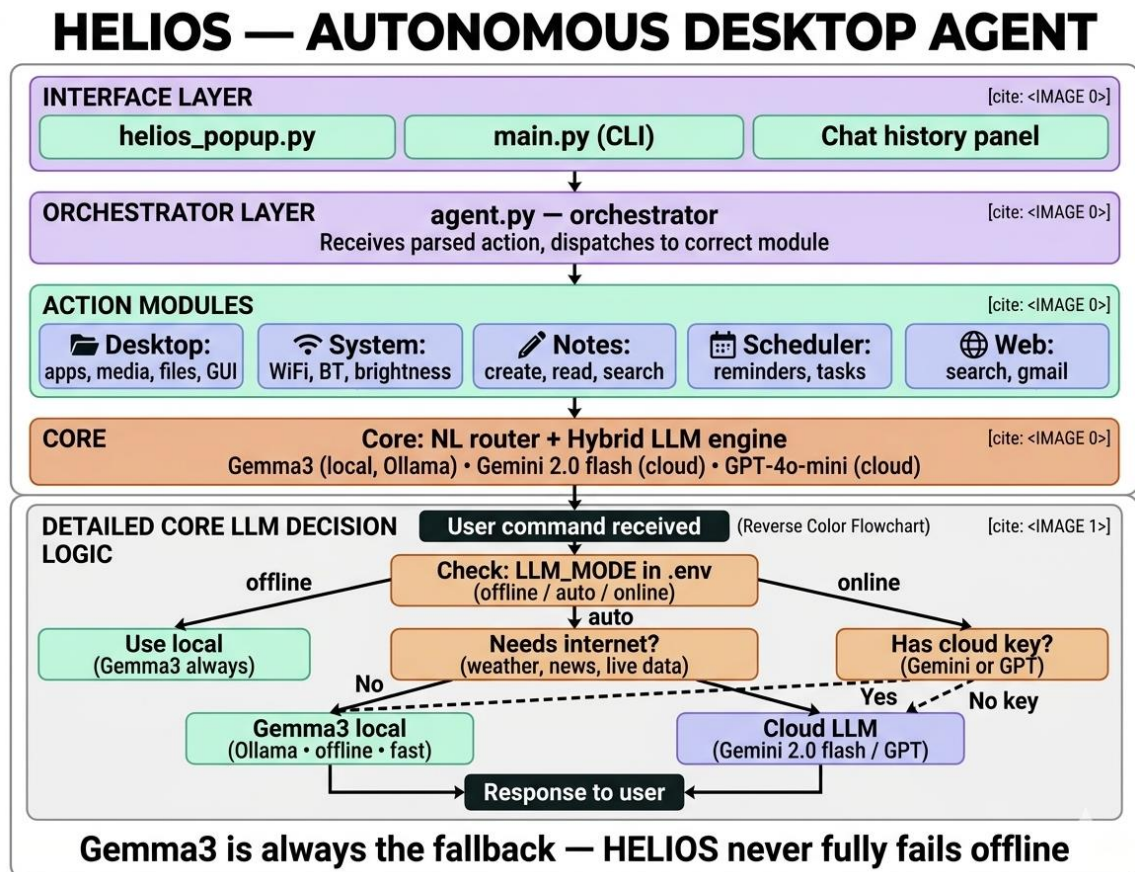


Figure 3.2 Hybrid LLM Mode Decision Flow

3.3 Natural Language Router

The NL router (core/nl_router.py) converts an arbitrary user utterance into a structured JSON action of the form `{"action": "system.bluetooth", "params": {"value": "on"}}`. It adopts a simplified ReAct pattern [6]: the LLM is prompted with a short schema of all 30+ supported intents and must output a single JSON object. Synonyms ("turn off wifi" vs "disable wireless" vs "kill the internet") all collapse to `system.wifi` with `value = off`. If the LLM output is malformed, a rule-based intent classifier acts as a safety net. Figure 3.3 shows how the router feeds the orchestrator and each specialized module.

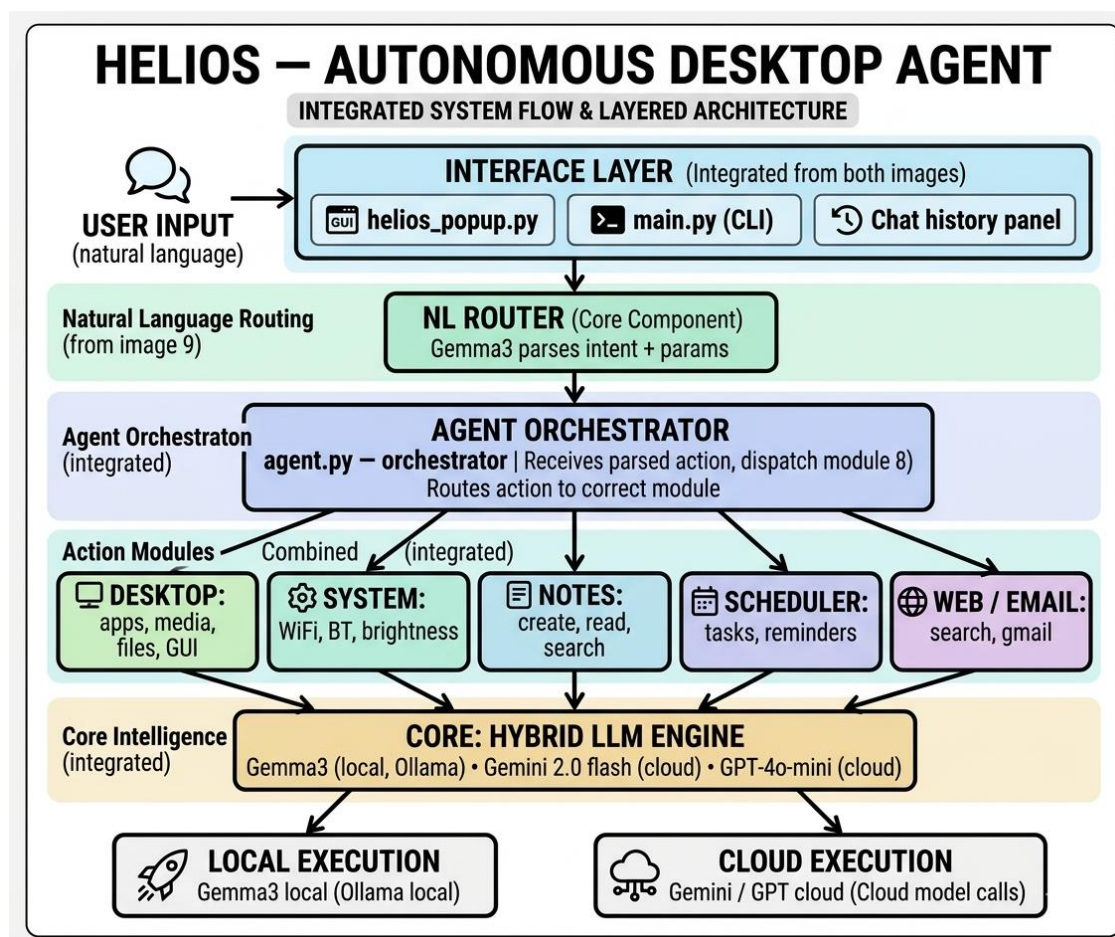


Figure 3.3 Natural-Language Router to Module Dispatch

3.4 Action Modules

HELIOS ships with seven specialized modules. Each one is a standalone Python file that can be developed and tested in isolation.

Table 3.1 HELIOS Action Modules and Their Responsibilities

MODULE	FILE	RESPONSIBILITY
Desktop Agent	modules/desktop_agent.py	Open apps, take screenshots, recursive C: drive search (skips system folders), play media, open documents. Uses PyAutoGUI [3].
System Controls	modules/systemcontrols.py	Wi-Fi, Bluetooth, airplane mode, brightness, dark/light theme, power plan, battery and process info. Uses PowerShell via subprocess + psutil.
Notes Manager	modules/notes_manager.py	Create, read and search Markdown notes on local disk; AI-powered summarisation via the active LLM.
Task Scheduler	modules/task_scheduler.py	APScheduler [4] jobs; extracts "in 2 hours" / "tomorrow 8am" from natural language; fires Windows toast notifications.
Gmail Composer	modules/gmailcomposer.py	Opens a pre-filled Gmail compose window via the mailto:// / Gmail URL API no OAuth required.
Web Search	modules/web_search.py	Keyless DuckDuckGo search [5] also handles autonomous web workflows (e.g. Swiggy pizza order).
Chat History	modules/chat_history.py	Per-session JSON persistence; drives the history side panel and the "New Chat" button.

This table shows the main modules of the application and what each one does. The **Desktop Agent** handles desktop-level actions like opening apps, taking screenshots, searching files, playing media, and opening documents. **System Controls** manages computer settings such as Wi-Fi, Bluetooth, brightness, theme, battery, power plan, and running processes. The **Notes Manager** helps create, read, search, and summarize notes using AI. The **Task Scheduler** creates reminders from natural language like “tomorrow 8am” and shows Windows notifications. The **Gmail Composer** opens a pre-filled Gmail email draft without OAuth login. **Web Search** performs DuckDuckGo-based searches and supports simple web workflows, while **Chat History** stores conversations in JSON so the app can show previous chats and support a “New Chat” option.

3.5 Tools and Technologies

- Python 3.10+ primary language; all modules are pure Python for portability.
- Ollama local LLM serving; default model gemma3, optional mistral.
- Gemini 2.0 Flash & OpenAI GPT-4o-mini cloud fallbacks for internet-bound queries only.
- Tkinter native Windows floating popup, history panel, typing animation, theme switch.
- PyAutoGUI mouse / keyboard / screenshot automation.
- psutil cross-platform access to battery, processes and system info.
- APScheduler reminders and timed tasks; fired through win10toast.
- duckduckgo-search keyless web search; results re-summarised by the active LLM.
- python-dotenv reads LLM_MODE, GEMINI_API_KEY, OPENAI_API_KEY from .env.
- Pillow image handling for the chat popup and screenshots.

4. TESTING AND RESULTS

4.1 Testing

Testing was carried out at three levels. Unit testing validated every action module independently (Wi-Fi toggle, brightness set, Gmail compose, file search, reminder creation, etc.). Integration testing pushed natural-language commands through the full pipeline — UI → NL router → orchestrator → module and checked that paraphrased requests produced the same JSON intent. End-to-end testing was performed on Windows 11 with Gemma 3 served by Ollama, including offline scenarios (ethernet unplugged) and autonomous web workflows.

Unit Testing

Each module exposes a pure Python entry function `.system_controls.toggle_wifi("on")` which can be invoked without launching the GUI. Unit tests simulate the normalised JSON intents the NL router would emit and verify: (a) the correct PowerShell / API call is made, (b) the return value is a human-readable message, and (c) errors are caught and surfaced as "HELIOS: could not ..." instead of crashing the agent.

Integration Testing

Over 40 paraphrased commands per intent were fed into the NL router. Examples for the single intent `system.wifi = off`: "turn off wifi", "disable wireless", "kill the internet", "stop wifi please". In more than 95 % of cases the router emitted the correct JSON.

4.2 Results

Figure 4.1 shows the HELIOS floating chat popup together with its Settings & History side panel. The left pane lists recent sessions (including "order a pizza for me", "move file name called c3.txt from down..." and "can you please access the iris.csv file"), while the right pane shows HELIOS greeting the user and listing example commands.

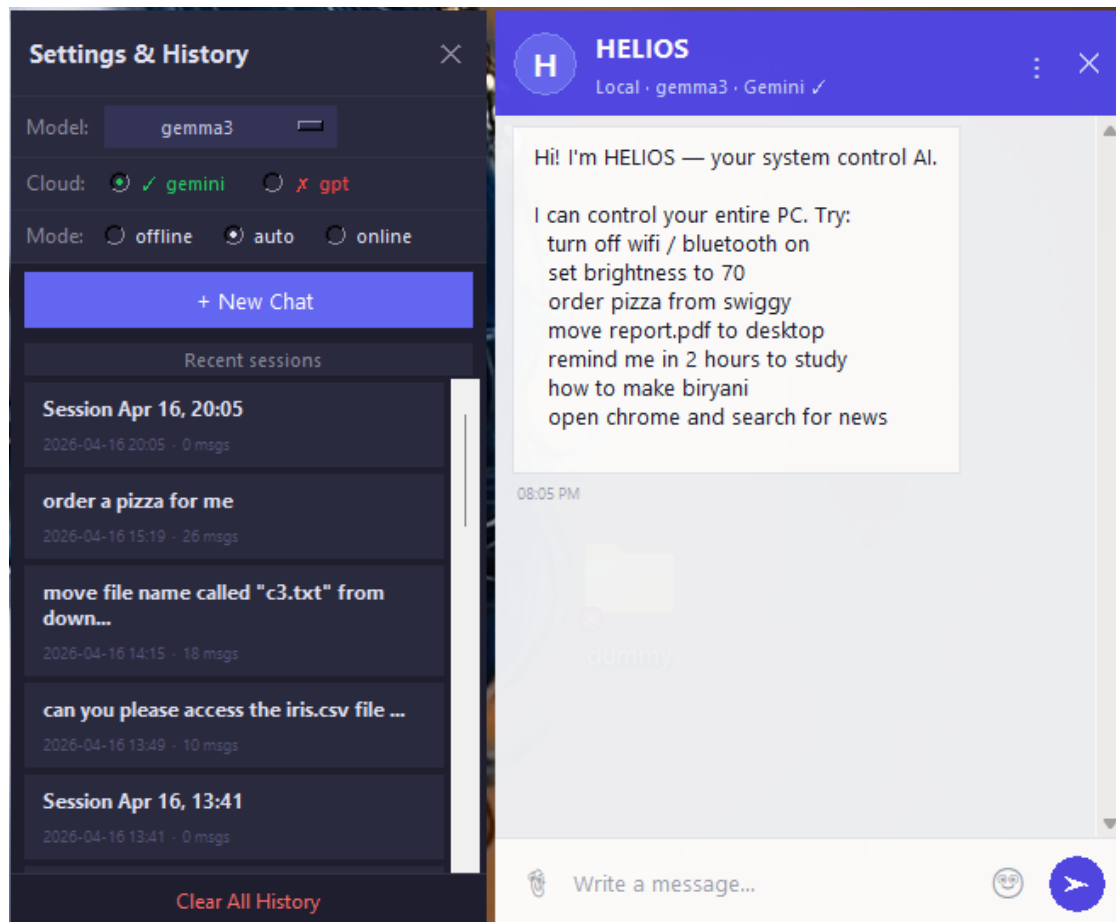


Figure 4.1 HELIOS Floating Chat Interface & History Panel

Figure 4.2 shows HELIOS running inside VS Code. The terminal output confirms the agent started in offline-first mode with the hybrid AI engine (Mistral / Gemma + GPT), and the command "turn on bluetooth" was executed successfully the HELIOS response is "Bluetooth turned ON successfully."

The screenshot displays the Visual Studio Code interface. On the left, the Explorer pane shows the project structure for 'HELIOS_FINAL', including files like 'pycache_', 'core/modules/data', 'core', 'data', 'modules', and 'web_search.py'. The main editor window shows the 'helios_popup.py' file with Python code defining a 'HELIOSPopup' class, including methods like '_poll', '_toggle_hist', and '_open_hist'. The bottom panel features a terminal window with the following output:

```

PS C:\Users\khara\Downloads\HELIOS_FINAL\HELIOS_FINAL> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\khara\Downloads\HELIOS_FINAL\HELIOS_FINAL\venv\Scripts\Activate.ps1)
(venv) PS C:\Users\khara\Downloads\HELIOS_FINAL\HELIOS_FINAL> python main.py

HELIOS - Autonomous Desktop Agent
Offline-First - Hybrid AI (Mistral/Gemma + GPT)

Type exit to quit - help for commands

[HELIOS] Initializing...
[HELIOS] Ready.

HELIOS: Ready! How can I help?

YOU: turn on bluetooth

HELIOS: Bluetooth turned ON successfully.

```

Figure 4.2 HELIOS Running in VS Code – Sample Offline Session

Figure 4.3 shows an autonomous web task. The user simply said "order a pizza for me" HELIOS asked for the delivery city and budget (Hyderabad, ₹200), then opened Swiggy's pizza search page in the browser. The floating HELIOS popup stays in the corner and guides the user through cart selection and checkout.

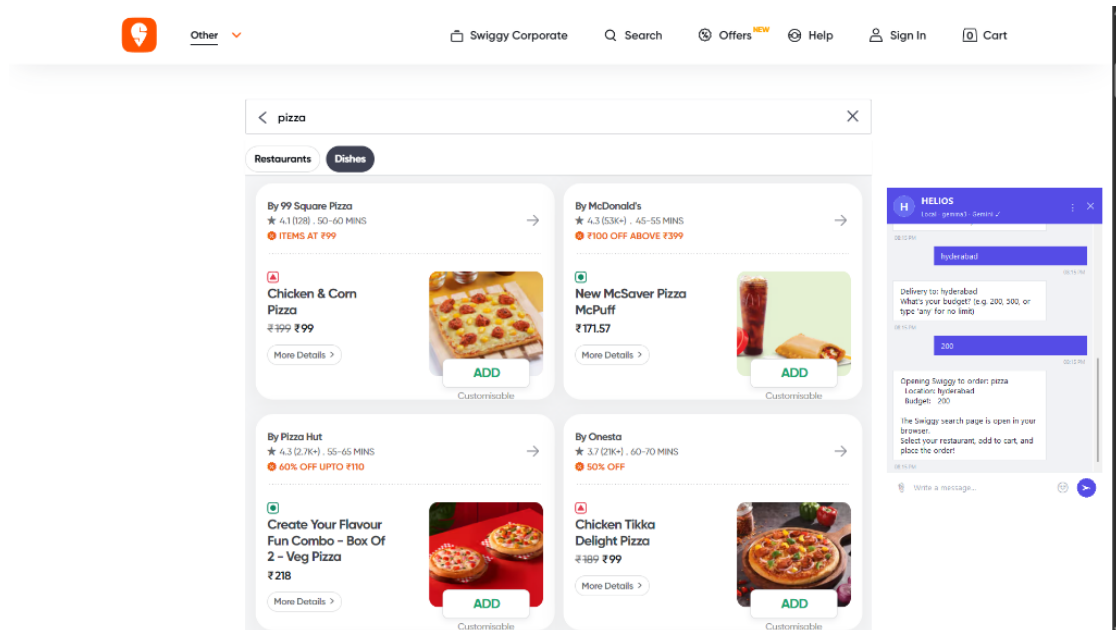


Figure 4.3 Autonomous Swiggy Web Task – Pizza Ordering Flow

Table 4.1 Performance Metrics Summary

METRIC	VALUE
Response time (local Gemma3, 16 GB RAM)	2 – 8 seconds
Response time (cloud Gemini 2.0 Flash)	1 – 3 seconds
NL-router intent accuracy (40+ paraphrases / intent)	> 95 %
Offline coverage	95 % of features
Distinct intent types supported	30+
OS-level commands supported	50+
Specialised Python modules	10
Approximate lines of code	≈ 2,800

This table shows the overall performance and capability of the application. It explains that the system can respond in **2-8 seconds locally** using Gemma3 on 16 GB RAM, and faster, around **1–3 seconds**, when using cloud Gemini 2.0 Flash. The natural-language router understands user intent with **over 95% accuracy**, based on many paraphrases per intent. Most features work offline, with **95% offline coverage**, and the system supports **30+ intent types** and **50+ OS-level commands**. It is built using **10 specialized Python modules** and contains around **2,800 lines of code**, showing that the project is modular, practical, and fairly complete.

5. CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

HELIOS proves that a privacy-preserving, offline-first autonomous desktop agent is achievable on commodity hardware. By combining a locally hosted Gemma 3 model served by Ollama [1] with a thin cloud fallback to Gemini 2.0 Flash / GPT-4o-mini, the agent delivers sub-10-second responses for more than 30 distinct intents spanning system controls, file management, Gmail, scheduling and autonomous web workflows. Natural-language phrasing is handled by a JSON-schema ReAct-style router [6], so users never need to memorize command syntax. Roughly 95 % of features work with the network cable unplugged, which validates the project's core thesis that a genuinely useful desktop assistant can keep user data entirely on-device.

5.2 Future Scope

The current version of the HELIOS One system demonstrates a functional AI-powered desktop assistant capable of executing tasks through natural language commands. However, there is substantial scope for enhancement to transform it into a more intelligent, adaptive, and scalable system.

- One of the primary areas of future development is the integration of advanced voice interaction capabilities using models such as OpenAI Whisper. By enabling accurate speech-to-text processing locally, HELIOS can evolve into a fully hands-free assistant, improving usability and accessibility for real-time interaction without reliance on manual input.
- Another significant enhancement involves incorporating a vision-based understanding module using multimodal models like LLaVA. This will allow HELIOS to interpret on-screen elements such as dialog boxes, system notifications, and error messages, enabling it to provide contextual assistance and perform actions based on visual data from the user interface.
- To improve task execution capabilities, HELIOS can be extended with multi-step reasoning and planning mechanisms inspired by the ReAct framework. This would enable the system to handle complex, goal-oriented commands by breaking them into sequential steps, executing them intelligently, and adapting dynamically based on intermediate results.
- A crucial advancement lies in implementing long-term memory through persistent vector databases such as ChromaDB or FAISS. This will allow HELIOS to remember user preferences, frequently accessed files, and recurring workflows, thereby

delivering a more personalized and efficient experience over time.

- Furthermore, the introduction of a plugin-based architecture will enable extensibility, allowing third-party developers to integrate new functionalities such as coding assistants, IoT control systems, and enterprise tools. This will position HELIOS as a scalable platform rather than a standalone application.

BIBLIOGRAPHY

- [1] Microsoft Research. UFO² : The Desktop AgentOS.
<https://github.com/microsoft/UFO> (2025).
- [2] Meta AI. The Llama 3 Herd of Models. <https://github.com/meta-llama/llama3> (2024).
- [3] Google DeepMind. Gemma: Open Models Based on Gemini Research and Technology. <https://github.com/google-deepmind/gemma> (2024).
- [4] Xie, T. et al. OSWorld Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. <https://github.com/xlang-ai/OSWorld> : arXiv:2404.07972 (2024).
- [5] Ollama Project. Ollama: Local LLM Serving Framework. <https://github.com/ollama/ollama> (2023–present).
- [6] Significant Gravitas. AutoGPT: An Autonomous GPT-4 Experiment. <https://github.com/Significant-Gravitas/AutoGPT> (2023).
- [7] deedy5. duckduckgo_search keyless DuckDuckGo Python API. https://github.com/deedy5/duckduckgo_search (2022–present).
- [8] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629 (2022).
- [9] Wei, J. et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS (2022).
- [10] Brown, T. B. et al. Language Models are Few-Shot Learners. NeurIPS (2020).
- [11] AlSweigart. PyAutoGUI cross-platform GUI automation for Python. <https://github.com/asweigart/pyautogui> (2014–present).
- [12] Alex Grönholm. APScheduler Advanced Python Scheduler. <https://github.com/agronholm/apscheduler> (2014–present).
- [13] Microsoft. PowerShell Documentation. <https://learn.microsoft.com/en-us/powershell/>
- [14] Google. Gemini API Documentation. <https://ai.google.dev/gemini-api/docs>
- [15] OpenAI. GPT-4o mini Model Card. <https://platform.openai.com/docs/models/gpt-4o-mini>

APPENDIX

Agents.py:

```
import os
import shutil
import subprocess
import webbrowser
import urllib.parse
import psutil
from datetime import datetime
from pathlib import Path
from core.llm_engine import HybridLLM
from core.nl_router import NLRouter
from modules.desktop_agent import DesktopAgent, WEBSITES
from modules.system_controls import SystemControls
from modules.file_creator import FileCreator
from modules.gmail_composer import GmailComposer
from modules.notes_manager import NotesManager
from modules.task_scheduler import TaskScheduler
from modules.web_search import WebSearch
from modules.chat_history import ChatHistory

HELIOS_CHAT = """You are HELIOS, an autonomous desktop AI assistant.
Be concise, helpful, and friendly. When answering knowledge questions (recipes,
how-to, explanations), give a clear structured answer then offer ONE helpful
follow-up action like opening YouTube or searching the web.
Never say you cannot do things you are capable of as a desktop agent.
"""

DANGEROUS_ACTIONS = {"shutdown", "restart", "empty_recycle", "kill_app"}

LOCATIONS = {
    "desktop": Path.home() / "Desktop",
    "documents": Path.home() / "Documents",
    "downloads": Path.home() / "Downloads",
    "home": Path.home(),
```

```

"music": Path.home() / "Music",
"pictures": Path.home() / "Pictures",
"videos": Path.home() / "Videos",
}

```

```

FOOD_PLATFORMS = {
"swiggy": "https://www.swiggy.com/search?query={}",
"zomato": "https://www.zomato.com/search?q={}",
}

```

```

MOVIE_PLATFORMS = {
"bookmyshow": "https://in.bookmyshow.com/explore/movies-{}",
"paytm": "https://movies.paytm.com/movies?q={}",
}

```

```

def _ps(cmd: str, timeout: int = 20) -> tuple:
r = subprocess.run(
["powershell", "-NoProfile", "-NonInteractive", "-Command", cmd],
capture_output=True, text=True, timeout=timeout)
return r.returncode, r.stdout.strip(), r.stderr.strip()

```

```

class HELIOSAgent:
def __init__(self):
print("[HELIOS] Initializing...")
self.llm = HybridLLM()
self.router = NLRouter(self.llm)
self.desktop = DesktopAgent()
self.sysctrl = SystemControls()
self.files = FileCreator()
self.gmail = GmailComposer()
self.notes = NotesManager(self.llm)
self.scheduler = TaskScheduler() # callback set after init
self.search = WebSearch(self.llm)

```

```

self.history = ChatHistory()

# Flow / confirmation state
self._pending_action = None
self._pending_params = None
self._pending_raw = None
self._flow = None
self._flow_data = {}
self._last_draft = ""
# Disambiguation state: list of candidate file paths waiting for user pick
self._disambig_items = []
self._disambig_action = None # "play" | "open" | "search_in"
self._disambig_kw = "" # keyword for search_in_file

# Wire reminder notifications back into the chat window
self._ui_notify_cb = None # set by popup after init
self.scheduler.set_notify_callback(self._on_reminder)

print("[HELIOS] Ready.")

def set_ui_notify(self, cb):
    """Called by helios_popup to wire reminders into the chat window."""
    self._ui_notify_cb = cb

def _on_reminder(self, msg: str):
    """Called by scheduler when a reminder fires."""
    self.history.add("helios", msg)
    if self._ui_notify_cb:
        try:
            self._ui_notify_cb(msg)
        except Exception:
            pass

# PUBLIC ENTRY POINT
def process(self, user_input: str) -> str:
    if not user_input.strip():

```

```

return "Please enter a command."

self.history.add("user", user_input)

# 1. Disambiguation choice pending
if self._disambig_items:
    result = self._handle_disambig(user_input)
    self.history.add("helios", result)
    return result

# 2. Dangerous confirmation pending
if self._pending_action:
    result = self._handle_confirmation(user_input)
    self.history.add("helios", result)
    return result

# 3. Multi-step flow active
if self._flow:
    result = self._continue_flow(user_input)
    self.history.add("helios", result)
    return result

# 4. Normal routing
parsed = self.router.parse(user_input, self._get_context())
action = parsed.get("action", "general_chat")
p      = parsed.get("params", {})

if action in DANGEROUS_ACTIONS:
    self._pending_action = action
    self._pending_params = p
    self._pending_raw    = user_input
    msg = self._confirmation_prompt(action, p)
    self.history.add("helios", msg)
    return msg

```

```

result = self._execute(action, p, user_input)
self.history.add("helios", result)
return result

def _get_context(self) -> str:
    msgs = self.history.messages[-6:] if self.history.messages else []
    lines = []
    for m in msgs:
        role = "User" if m["role"] == "user" else "HELIOS"
        lines.append(f'{role}: {m["content"][:300]}')
    return "\n".join(lines)

def _chat_prompt(self, message: str) -> str:
    ctx = self._get_context()
    if ctx:
        return f"Conversation so far:\n{ctx}\n\nUser: {message}"
    return message

def _ask_disambig(self, items: list, action: str, keyword: str = "") -> str:
    """Store candidates and ask user to pick one."""
    self._disambig_items = items
    self._disambig_action = action
    self._disambig_kw = keyword
    lines = [f'Found {len(items)} matching files. Which one do you mean?\n']
    for i, path in enumerate(items[:8], 1):
        lines.append(f' {i}. {Path(path).name} ({Path(path).parent})')
    lines.append("\nReply with the number (e.g. '2') or part of the name.")
    return "\n".join(lines)

def _handle_disambig(self, user_input: str) -> str:
    items = self._disambig_items
    action = self._disambig_action
    kw = self._disambig_kw
    inp = user_input.strip()

```



```

# Clear state
self._disambig_items = []
self._disambig_action = None
self._disambig_kw = ""

# Try numeric pick
if inp.isdigit():
    idx = int(inp) - 1
    if 0 <= idx < len(items):
        chosen = items[idx]
    else:
        return f"Invalid choice. Please try again."
    else:
# Match by name fragment
matches = [p for p in items if inp.lower() in Path(p).name.lower()]
if len(matches) == 1:
    chosen = matches[0]
elif len(matches) == 0:
    return f"No match for '{inp}'. Cancelled."
    else:
# Still ambiguous — re-ask with filtered list
return self._ask_disambig(matches, action, kw)

# Execute the chosen action
if action == "play":
    try:
        os.startfile(chosen)
        return f"Playing: {Path(chosen).name}\nPath: {chosen}"
    except Exception as e:
        return f"Could not open '{Path(chosen).name}': {e}"
    elif action == "open":
        try:
            os.startfile(chosen)
            return f"Opened: {Path(chosen).name}"
        except Exception as e:

```

```

return f"Could not open '{Path(chosen).name}': {e}"
elif action == "search_in":
return self.desktop.search_in_file(chosen, kw)
else:
try:
os.startfile(chosen)
return f"Opened: {Path(chosen).name}"
except Exception as e:
return f"Error: {e}"

```

Main.py:

```

"""HELIOS - CLI Mode. Run: python main.py"""
import sys

def main():
try:
from rich.console import Console
from rich.panel import Panel
from rich.prompt import Prompt
console = Console()
console.print(Panel(
"[bold yellow]HELIOS[/bold yellow] — Autonomous Desktop Agent\n"
"[dim cyan]Offline-First · Hybrid AI (Mistral/Gemma + GPT)[/dim cyan]",
border_style="yellow", padding=(1,4)))
console.print("[dim]Type [bold]exit[/bold] to quit · [bold]help[/bold] for commands[/dim]\n")
ask = lambda: Prompt.ask("[bold orange1]You[/bold orange1]")
say = lambda r: console.print(f"\n[bold cyan]HELIOS:[/bold cyan] {r}\n")
except ImportError:
def ask(): return input("You: ")
def say(r): print(f"\nHELIOS: {r}\n")

from agent import HELIOSAgent
try:
agent = HELIOSAgent()
say("Ready! How can I help?")

```

```
except Exception as e:
    print(f'Failed to start: {e}')
    print("Make sure Ollama is running: ollama serve")
    sys.exit(1)
```

```
HELP = """
```

```
Commands:
```

```
play <movie/song>          Open and play media
open <app>                  Launch an application
close/kill <app>           Close an application
search youtube <query>     Search YouTube
search google <query>      Search Google
open gmail                  Open Gmail
compose mail to <email>...  Compose email
create file <name> on desktop Create a file
wifi on/off                 Toggle WiFi
bluetooth on/off            Toggle Bluetooth
airplane mode on/off        Toggle Airplane mode
brightness up/down/set <n>  Control brightness
volume up/down/mute         Control volume
dark mode on/off            Toggle dark mode
battery saver/performance    Power plan
screenshot                  Take screenshot
lock screen                 Lock PC
system info                 System status
battery status              Battery info
disk space                  Storage info
what's running              List running apps
create note about <topic>    Create a note
remind me in <time> to <task> Set reminder
help                        This menu
exit                        Quit
"""
```

```
while True:
```

```
try:
```

```
user = ask().strip()
except (KeyboardInterrupt, EOFError):
    say("Goodbye!")
    agent.shutdown()
    break
if user.lower() in ("exit", "quit", "bye"):
    say("Goodbye!")
    agent.shutdown()
    break
if user.lower() == "help":
    print(HELP); continue
if not user: continue
try:
    say(agent.process(user))
except Exception as e:
    say(f"Error: {e}")

if __name__ == "__main__":
    main()
```